Static and Molecular-Dynamics-Derived Residue Graphs for Protein–Protein Binding Affinity Prediction

Student Name

The thesis is submitted to University College Dublin
in part fulfilment of the requirements for the degree of
BSc Applied and Computational Mathematics

School of Mathematics and Statistics
and
Supervisor's School (if different)
University College Dublin

Supervisor: Supervisor Name
Second Supervisor Name (if applicable)

March 18, 2026

Abstract

This thesis studies protein–protein binding affinity prediction from structure and asks one question: does a short explicit-solvent molecular-dynamics simulation improve a residue-graph affinity model over a strong static baseline? The pipeline starts from a curated wild-type subset of PPB-Affinity, prepares each complex for molecular dynamics, runs a short OpenMM simulation, constructs residue graphs with DeepRank2, adds trajectory-derived node and edge summaries, and evaluates either a masked graph variational autoencoder or a direct supervised graph regressor.

The starting processed table contains 2818 complexes. The final model matrix was run on a smaller compute-bounded 1437-complex analysis subset for which paired full and interface-restricted residue graphs were available. The matrix contains 52 configurations: 48 GraphVAE configurations and 4 supervised baselines. Each configuration is evaluated over 10 outer held-out fits from 5 folds and 2 repeats.

The strongest model is the direct supervised baseline on the full static graph, with mean test RMSE 2.3379, Pearson correlation 0.5064, and R^2 0.2146. Within the GraphVAE family, the best configuration is the full static semi-supervised model with latent dimension 32, with mean test RMSE 2.4292. The full graph outperforms the interface-restricted graph in all 24 matched GraphVAE comparisons. Under the shared-static reconstruction target, the static representation outperforms the static-plus-dynamic representation in 11 of 12 matched GraphVAE comparisons.

For this pipeline, the molecular-dynamics-derived summaries do not improve held-out affinity prediction over a static residue graph. The best-performing representation among the tested models is the full static residue graph.

Acknowledgments

I am grateful to my supervisor for guidance throughout the project and for repeated feedback on both the technical work and the written thesis. I also thank the staff of the School of Mathematics and Statistics and the collaborating research environment that made the computational work possible. Finally, I thank my family and friends for their support during the project.

Contents

1 Introduction **1**

2 Literature Review **2**

2.1 Protein–protein affinity prediction as a benchmark problem 2

2.2 Physics-based, empirical, and learned approaches 3

2.3 Generalization and evaluation 4

2.4 Why use a variational model 4

2.5 Position of this thesis 4

3 Methods **5**

3.1 Dataset and target 5

3.2 Molecular-dynamics preparation and simulation 5

3.3 Residue-graph construction and features 6

3.4 Graph views, analysis subset, and preprocessing 7

3.5 Models 8

3.5.1 Direct supervised baseline 8

3.5.2 Masked GraphVAE 8

3.5.3 Latent-space ridge probe 9

3.6 Repeated outer evaluation 9

4 Results **11**

4.1 Headline models 11

4.2 Factor comparisons 11

4.3 Family-level train, validation, and test behaviour 12

4.4 Performance by interaction class 12

4.5 Summary 13

5 Discussion and Conclusions **14**

A Archived Approaches and Project Evolution **16**

A.1 Correction learning around fast affinity baselines 16

A.1.1 PRODIGY 16

A.1.2 MM/GBSA 17
A.2 The Cα contact-map branch 17
A.3 The PPB-inspired augmentation and spatiotemporal branch 19
A.4 Exploratory dataset curation utilities 19

B Complete Model Matrix **20**

C Repository Layout **23**

List of Figures

List of Tables

2.1 Representative reported results from the protein–protein and protein–ligand affinity literature. Reported correlations are taken from the original papers; the datasets differ across rows. 3

3.1 Feature groups used in the residue-graph pipeline. The static mode S used only the static block. The dynamic mode SD used the union of the static and dynamic blocks. 7

3.2 Masked reconstruction targets used by the GraphVAE. All S runs used `shared_static`. All SD runs used `shared_static`, `sd_dynamic_all`, or `sd_static_plus_dynamic_all`. 9

4.1 Best-performing models in the final matrix. GraphVAE metrics are those of the downstream ridge probe on latent means. 12

4.2 Matched GraphVAE comparisons. A positive RMSE gain means the first-named option has lower mean test RMSE. 12

4.3 Family-level mean train, validation, and test behaviour. Here $g_r = r_{\text{train}} - r_{\text{test}}$. Each row is averaged over all configurations in that family. 13

4.4 Pooled outer-test subgroup metrics for the best overall model (`full_base_S`), the best GraphVAE (`full_S_semi_shared_static_z32`), and the best interface model (`iface_base_S`). 13

A.1 Saved subset-specific PRODIGY results from archived curated datasets. The TCR-pMHC subset was especially weak, while the small antibody–antigen subset was better but still modest. 17

A.2 Representative archived PRODIGY-based regression runs on the TCR subset. Direct latent regression reduced RMSE relative to raw PRODIGY, but the correction-to-PRODIGY route was not competitive. 17

A.3 MM/GBSA shard results from the archived branch. The estimates are numerically far outside the scale of experimental protein–protein binding free energies, so the branch was not usable as a quantitative baseline. . . . 18

A.4 Representative CA contact-map regression runs on the archived TCR subset. The branch could achieve lower RMSE than raw PRODIGY on some runs, but the results were unstable and often had weak or negative test correlation. 18

B.1 Complete 52-model evaluation matrix, sorted by mean test RMSE. Metrics are means over 10 outer held-out fits. 21

Chapter 1

Introduction

Protein–protein binding affinity prediction asks for a quantitative map from structure to binding free energy. The problem is important in molecular recognition and protein design, and it is difficult because the target is thermodynamic while the input is usually a single experimental structure [1–3]. This thesis studies that problem on a curated subset of PPB-Affinity [3].

The thesis asks whether a short explicit-solvent molecular-dynamics simulation provides useful predictive information beyond a static residue graph. It evaluates one implemented residue-graph pipeline. Each complex is prepared for molecular dynamics, simulated in OpenMM, converted to a residue graph with DeepRank2, augmented with trajectory-derived summaries, and then evaluated with either a masked GraphVAE or a direct supervised graph regressor. The processed broad table contains 2818 complexes, but the final comparison uses a smaller compute-bounded 1437-complex analysis subset for which paired full and interface-restricted graph views are available.

The implementation, curated inputs, and compact result summaries are available in the repository `https://github.com/RohanDevereux/Bridging`. Appendix A records the archived branches explored before the final pipeline was fixed.

The thesis asks four empirical questions. Is a full residue graph better than an interface-restricted representation? Do the molecular-dynamics-derived summaries improve prediction over static graph features alone? Does a masked GraphVAE yield a better predictive representation than a direct supervised graph regressor with the same encoder backbone? How stable are the answers under repeated held-out evaluation rather than a single train/test split?

Chapter 2 places the work in the literature on protein–protein affinity prediction, molecular simulation, and learned structural representations. Chapter 3 describes the implemented pipeline. Chapter 4 reports the final model matrix. Chapter 5 gives the main conclusions. The appendices cover archived branches, the complete 52-model matrix, and the repository layout.

Chapter 2

Literature Review

2.1 Protein–protein affinity prediction as a benchmark problem

Absolute protein–protein affinity prediction is a smaller and less standardized literature than protein–ligand scoring. Protein–ligand benchmarking has had a shared structural resource since PDBbind [4], whereas curated protein–protein benchmarks for absolute affinity remained much smaller for many years [5]. PPB-Affinity is a recent dataset built specifically for machine-learning work on direct protein–protein affinity prediction at larger scale [3]. Reviews of the field still describe limited labelled data, broad interface heterogeneity, and conformational change as central obstacles [2, 3, 6].

Single-structure scoring is particularly restrictive for protein–protein complexes. In the Kastritis benchmark, only a minority of complexes were close to rigid-body association, and large or moderate interface rearrangements were common [5]. Moal et al. likewise emphasized that flexible complexes and diverse functional classes make the task harder than the narrow enzyme–inhibitor benchmarks used by much of the older literature [7]. The literature is correspondingly more fragmented than the protein–ligand literature.

PPB-Affinity also makes the subtype heterogeneity explicit. Its benchmark model reached Pearson $r = 0.701$ on the full test set, but the subgroup plots were weaker for both antibody–antigen and TCR-pMHC [3]. The paper attributes the TCR-pMHC difficulty to strong interface similarity within that class and notes that both subgroups contain a higher proportion of mutant samples than the full dataset [3]. Subgroup-specific models have appeared for the same reason. Yang et al. developed antibody–antigen-specific affinity regressors and argued that general protein–protein affinity predictors do not transfer to antibody–antigen complexes with comparable accuracy [8].

Domain	Method	Representation / scope	Model class	Reported result
Protein–protein	PRODIGY [13, 14]	Interfacial contacts and non-interacting surface descriptors; interface-only	Empirical regression	Benchmark set of 79 complexes: $r = 0.74$, MAE 1.4 kcal mol^{-1}; external 90-complex set reported later by PPI-Affinity: $r = 0.31$, MAE 2.5 kcal mol^{-1}
Protein–protein	PPI-Affinity [14]	Structure-derived descriptor vector; whole-complex descriptors	Ensemble regressor	Vangone–Bonvin benchmark: $r = 0.62$, MAE 1.8 kcal mol^{-1}; external 90-complex hold-out: $r = 0.50$, MAE 1.8 kcal mol^{-1}
Protein–protein	PPB-Affinity benchmark model [3]	128 interface residues and pair geometry; interface-restricted	IPA-based geometric deep learning	Whole-dataset benchmark: Pearson $r = 0.701$, Spearman $\rho = 0.691$
Protein–protein	PPI-Graphomer [15]	Pretrained sequence/structure features with interface-aware bias; interface-focused	Graph transformer	Two external test sets: PCC 0.641 and 0.625
Protein–ligand	Pafnucy [16]	3D voxelized binding-site tensor; pocket-restricted	3D CNN	PDBbind v2016 core-set scoring power: Pearson $r = 0.78$
Protein–ligand	Timenucy / Videonucy [17]	MD trajectories of protein–ligand complexes; pocket-region time series	Spatio-temporal neural networks	MDbind test set: Pearson $r = 0.78$ for Timenucy and $r = 0.84$ for Videonucy

Table 2.1: Representative reported results from the protein–protein and protein–ligand affinity literature. Reported correlations are taken from the original papers; the datasets differ across rows.

2.2 Physics-based, empirical, and learned approaches

Physics-based affinity estimation remains the reference point. Umbrella sampling and alchemical free-energy methods target the binding free energy more directly, but they are expensive and sensitive to protocol design [9, 10]. MM/PBSA and MM/GBSA are cheaper endpoint approximations and are therefore attractive as practical baselines, but they remain approximate and system-dependent [11, 12]. Fast empirical estimators occupy the other extreme. PRODIGY predicts affinity from interfacial contact counts and non-interacting surface descriptors [13], while descriptor-based regressors such as the model of Moal et al. use engineered structural summaries of the complex [7].

More recent work has moved toward learned structural representations. In protein–protein affinity prediction, the common pattern is still interface-centered learning: hand-designed interface descriptors, interface-derived descriptor vectors, or interface-focused geometric deep learning [3, 14, 15]. In protein–ligand affinity prediction, the literature is broader and older, spanning binding-site voxel CNNs such as Pafnucy [16] and MD-aware spatio-temporal models such as Timenucy and Videonucy [17].

2.3 Generalization and evaluation

External-test-set behavior already exposed weak generalization in this literature. PPI-Affinity reported that PRODIGY fell from $r = 0.74$ on the original benchmark of Vangone and Bonvin to $r = 0.31$ on a separate 90-complex hold-out set, and interpreted that drop as evidence of overfitting to the earlier benchmark [14]. Moal et al. had already tried to guard against the same problem by removing homologous complexes so that leave-one-out evaluation would not benefit from near-duplicate training examples [7]. In protein–ligand scoring, Graber et al. later showed directly that structural overlap between PDBbind and CASF had inflated apparent benchmark performance, and that retraining on leakage-reduced splits caused marked performance drops for previously strong models [18]. External or family-disjoint evaluation is necessary when reported accuracy is interpreted.

2.4 Why use a variational model

Variational autoencoders were introduced as latent-variable models that combine reconstruction with a regularized latent distribution [19]. Variational graph autoencoders extended the same idea to graph data [20]. That is not yet a standard route for endpoint protein–protein affinity prediction, but it is relevant as a representation-learning strategy when the input is high-dimensional and only part of the signal is likely to be affinity-relevant.

The specific motivation comes from molecular simulation. Tian et al. showed that a VAE trained on protein MD trajectories could learn a latent space that generated unsampled but physically plausible conformations and helped explore hidden regions of the conformational landscape [21]. The role of the GraphVAE in this thesis is analogous. It is not used as a generic graph generator; it is used to compress residue graphs built from static structure and short MD-derived summaries into a latent representation that can then be tested for affinity prediction.

2.5 Position of this thesis

This thesis sits between fast structure-based scoring and expensive free-energy calculation. It does not attempt rigorous binding free-energy estimation, and it does not rely on a small hand-designed interface score alone. The central question is whether short explicit-solvent MD can improve a residue-graph representation for direct protein–protein affinity prediction, and whether that answer changes with graph view or model family.

Chapter 3

Methods

3.1 Dataset and target

All experiments used a curated wild-type subset of PPB-Affinity [3]. The local curation retained one ordered ligand–receptor chain assignment per complex, removed duplicate entries, and excluded mutant complexes. A complex was kept only if a Protein Data Bank structure, an interpretable ligand/receptor chain definition, and an affinity measurement sufficient to derive a binding free energy were available. The resulting processed CSV contained 2818 complexes.

The interaction classes in this processed table were 199 TCR-pMHC complexes, 437 antibody–antigen complexes, and 2182 other or unannotated complexes. PPB-Affinity reports different affinity distributions and weaker benchmark performance for the TCR-pMHC and antibody–antigen subsets than for the full test set, so this heterogeneity is part of the benchmark rather than an afterthought [3].

The regression target was the standard Gibbs free energy of binding, ΔG, in kcal mol^{-1}. In this processed table, the affinity column used by the pipeline was the reported dissociation constant K_D. All targets were therefore computed as

$$\Delta G = RT \ln K_D, \tag{3.1}$$

with $R = 0.00198720425864083$ kcal mol^{-1} K^{-1} and the experimental temperature in kelvin. When no usable temperature value was available in the processed table, $T = 298.15$ K was used.

3.2 Molecular-dynamics preparation and simulation

Complexes were prepared from PDB coordinates with PDBFixer and the OpenMM application layer [22, 23]. The annotated ligand and receptor chains were extracted first. Missing residues were detected but not rebuilt. Nonstandard residues were converted

when a standard mapping was available, heterogens were removed, missing heavy atoms were added, and hydrogens were assigned before solvation.

Prepared complexes were solvated with 1.0 nm padding and ionic strength 0.15 M. The force-field stack was `amber14-all.xml` for the protein and `amber14/tip3pfb.xml` for the solvent [24, 25]. The processed PPB-Affinity table used here had no pH column, so protonation was carried out at pH 7.0 for every complex.

All simulations were run in OpenMM with PME electrostatics, a 1.0 nm real-space cutoff, constraints on bonds involving hydrogen, rigid water, a Langevin middle integrator with friction 1 ps^{-1}, and a 2 fs timestep [23]. Each system was minimized for up to 1000 iterations, equilibrated for 100,000 steps, and then simulated for 500,000 further steps. Reporters were written every 5000 steps. Because the reporters were attached before equilibration, each complex contributed 120 saved frames over 1.2 ns in total.

3.3 Residue-graph construction and features

Residue graphs were built with DeepRank2 [26]. Graph extraction used a full-complex query radius of 10^6 Å, so every protein residue in the prepared topology was retained. The DeepRank2 modules `components`, `contact`, `exposure`, and `surfacearea` supplied the basic residue and edge descriptors.

The final input features are listed in Table 3.1. Static features were present in both model modes. Dynamic features were added only in the SD mode.

Dynamic node and edge features were computed from the saved trajectories with MDTraj [27]. Hydration was measured from the full solvated trajectory by counting unique water oxygen residues within 5 Å of any heavy atom of a residue. The stored hydration features were $\log(1 + \bar{w}_i)$ and the standard deviation of the framewise water counts. The logarithm controls the right tail from highly solvent-exposed residues while preserving zero exactly. Cα RMSF was computed after least-squares alignment. Backbone torsions were summarized by frame-averaged values of $\sin\phi$, $\cos\phi$, $\sin\psi$, and $\cos\psi$.

Dynamic edge features were computed only on residue pairs already connected in the static graph. For each edge, the closest heavy-atom distance was evaluated in every saved frame. The dynamic contact frequency was the fraction of frames with closest-heavy distance at most 8 Å. When enabled, the mean and standard deviation of the closest-heavy distance were also stored.

Each residue also received approximate inter-chain nonbonded force summaries. A protein-only OpenMM analysis system was rebuilt from the saved protein topology using the same force-field files as the original simulation. The original nonbonded parameters were copied into separate Coulomb and Lennard–Jones `CustomNonbondedForce` objects and restricted to ligand-atom versus receptor-atom interaction groups [23]. For each saved frame, per-atom forces were accumulated to the residue level and reduced to Euclidean

Group	Node features	Edge features
Static	DeepRank2 residue descriptors `bsa`, `sasa`, `res_depth`, `hse_0`, `hse_1`, `hse_2`; local residue counts `n_same_chain_8A`, `n_other_chain_8A`; 20 amino-acid one-hot indicators; residue-type flags `aa_type_nonpolar`, `aa_type_polar`, `aa_type_charged`; physicochemical scalars `phys_sidechain_size`, `phys_net_charge`, `phys_hydrophobicity`, `phys_alpha_propensity`, `phys_beta_propensity`; terminus flags `is_n_terminus`, `is_c_terminus`	`distance`, `same_chain`, `electrostatic`, `vanderwaals`
Dynamic	Hydration and flexibility summaries `dyn_log1p_water_count_5A_mean`, `dyn_water_count_5A_std`, `dyn_rmsf_ca`; torsion summaries `dyn_sin_phi_mean`, `dyn_cos_phi_mean`, `dyn_sin_psi_mean`, `dyn_cos_psi_mean`; force-proxy summaries `dyn_coul_force_mean`, `dyn_coul_force_std`, `dyn_lj_force_mean`, `dyn_lj_force_std`	`dyn_contact_freq_8A`, `dyn_dist_mean`, `dyn_dist_std`

Table 3.1: Feature groups used in the residue-graph pipeline. The static mode S used only the static block. The dynamic mode SD used the union of the static and dynamic blocks.

norms. The stored features were the framewise means and standard deviations of those residue-level norms for the Coulomb and Lennard–Jones channels.

3.4 Graph views, analysis subset, and preprocessing

Two graph views were evaluated. The *full* view retained the complete residue graph. The *interface-restricted* view kept a local neighborhood around the closest ligand-residue/receptor-residue pair identified from mapped trajectory Cα coordinates, under a hard cap of 128 residues.

The final thesis did not run MD simulation, feature extraction, and repeated model evaluation on all 2818 curated complexes. Those stages were restricted to a smaller analysis subset for time and compute reasons. The final paired full/interface-restricted graph set contained 1437 complexes: 105 TCR-pMHC, 366 antibody–antigen, and 966 other complexes. A small additional number of complexes were lost during partner-chain reconciliation and view construction, but the reduction from 2818 to 1437 should be read primarily as a compute limitation rather than as a statement that the remainder of PPB-Affinity was structurally unusable. PPB-Affinity also reports that its benchmark model improves as the training fraction increases, so dataset scale is itself part of the modelling

problem [3].

Feature scaling was fit separately inside each outer split and only on that split's training complexes. Every numeric feature was clipped to its 0.5th and 99.5th training percentiles, standardized to zero mean and unit variance, and clipped again to $[-8, 8]$. The same fitted transformation was then applied to the validation and test complexes.

3.5 Models

3.5.1 Direct supervised baseline

The supervised baseline was a residue-graph regressor implemented in PyTorch Geometric [28]. Node and edge inputs were projected linearly to 128 hidden units, passed through three GINE layers, pooled with the concatenation of global mean pooling and global max pooling, and mapped to ΔG by a two-layer multilayer perceptron [29, 30]. The same encoder depth, hidden dimension, optimizer, batch size, and early-stopping settings were used for both graph views and both feature modes.

3.5.2 Masked GraphVAE

The GraphVAE used the same encoder backbone as the supervised baseline: linear node and edge projections to 128 hidden units, three GINE layers, and mean-plus-max graph pooling. Two linear heads mapped the pooled representation to the latent mean μ and log-variance $\log \sigma^2$. The latent dimensionality was one of $d_z \in \{8, 16, 32\}$.

For every batch, target node entries and target edge entries were masked independently with probability 0.30. The encoder received the masked graph together with binary mask indicators. The node decoder received the graph latent, the unmasked non-target node inputs, the masked target node inputs, and the node-mask indicators. The edge decoder received the analogous edge quantities. Reconstruction was therefore explicit and feature-specific rather than a generic graph-generation objective.

The reconstruction targets used in the final matrix are listed in Table 3.2.

For a batch of graphs, the GraphVAE objective was

$$\mathcal{L} = \mathcal{L}_{\text{node}} + \mathcal{L}_{\text{edge}} + \beta \mathcal{L}_{\text{KL}} + 0.01\, \mathcal{L}_{\text{corr}} + \lambda_{\text{aff}} \mathcal{L}_{\text{aff}}, \tag{3.2}$$

where $\mathcal{L}_{\text{node}}$ and $\mathcal{L}_{\text{edge}}$ were masked mean-squared reconstruction losses, $\mathcal{L}_{\text{KL}}$ was the standard Gaussian VAE penalty [19], $\mathcal{L}_{\text{corr}}$ penalized the squared off-diagonal entries of the latent covariance matrix, and $\mathcal{L}_{\text{aff}}$ was an MSE loss on normalized ΔG from a latent MLP head. Unsupervised runs used $\lambda_{\text{aff}} = 0$. Semi-supervised runs used $\lambda_{\text{aff}} = 1$.

All GraphVAE models were trained with AdamW, learning rate 10^{-4}, weight decay 10^{-5}, batch size 16, maximum 120 epochs, and early-stopping patience 20. The KL weight

Target policy	Reconstructed node features	Reconstructed edge features
shared_static	bsa, sasa, res_depth, hse_0, hse_1, hse_2, n_same_chain_8A, n_other_chain_8A	distance
sd_dynamic_all	dyn_log1p_water_count_5A_mean, dyn_water_count_5A_std, dyn_rmsf_ca, dyn_sin_phi_mean, dyn_cos_phi_mean, dyn_sin_psi_mean, dyn_cos_psi_mean, dyn_coul_force_mean, dyn_coul_force_std, dyn_lj_force_mean, dyn_lj_force_std	dyn_contact_freq_8A, dyn_dist_mean, dyn_dist_std when present
sd_static_plus_dynamic_all	Union of shared_static and sd_dynamic_all node targets	Union of distance and dynamic edge targets

Table 3.2: Masked reconstruction targets used by the GraphVAE. All S runs used shared_static. All SD runs used shared_static, sd_dynamic_all, or sd_static_plus_dynamic_all.

increased linearly from 0 to 1 over the first 30% of training epochs. Checkpoint selection used validation reconstruction loss for unsupervised runs and validation reconstruction loss plus weighted validation affinity loss for semi-supervised runs.

3.5.3 Latent-space ridge probe

GraphVAE results are reported from a linear probe on the latent means rather than from the neural affinity head alone. After training, each retained checkpoint exported graph-level latent means for the train, validation, and test complexes. A ridge regressor was then fit on the training latents with RidgeCV from scikit-learn [31] over

$$\alpha \in \{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}, 10^{-1}, 3 \times 10^{-1}, 1, 3, 10, 30, 100\}.$$

The headline GraphVAE metrics in Chapter 4 are the held-out metrics of this ridge probe.

3.6 Repeated outer evaluation

The final model matrix was evaluated on the matched 1437-complex set with 5 outer folds and 2 repeats. Complex identifiers, not trajectory frames, were the unit of resampling. In each outer fit, one fold was held out as test and the remaining four folds were split into training and validation sets to give an overall train/validation/test proportion of approximately 70/15/15. Every stage of the pipeline was repeated inside each outer split: feature scaling, model training, checkpoint selection, and latent ridge fitting.

The final matrix contained 52 configurations. Forty-eight were GraphVAE configura-

tions spanning two graph views (full and interface-restricted), two feature modes (S and SD), two supervision modes (unsupervised and semi-supervised), and latent dimensions 8, 16, and 32. The SD runs used three target policies; the S runs used only `shared_static`. Four further configurations were direct supervised baselines for the Cartesian product of graph view and feature mode.

Held-out performance was measured with RMSE, Pearson correlation, and R^2. Every metric reported in Chapter 4 is the arithmetic mean over the 10 outer fits for the corresponding configuration.

Chapter 4

Results

The final evaluation used the 1437-complex analysis subset for which paired full and interface-restricted graph views were available. The model matrix contained 52 configurations: 48 GraphVAE configurations and 4 direct supervised baselines. Every reported metric is the mean over 10 outer held-out fits from 5 folds and 2 repeats. The complete 52-model table is given in Appendix B.

4.1 Headline models

Table 4.1 lists the strongest models in the final matrix. The best overall model was the direct supervised baseline on the full static graph. The best GraphVAE was the full static semi-supervised model with latent dimension 32. The best interface model was the static supervised baseline.

The best supervised baseline outperformed the best GraphVAE by 0.0913 RMSE units and 0.0415 Pearson-correlation units.

4.2 Factor comparisons

Table 4.2 summarizes the main matched comparisons in the GraphVAE matrix. The dominant effect was graph view: the full graph outperformed the interface-restricted graph in all 24 matched GraphVAE comparisons. Under the `shared_static` target policy, the static representation outperformed the static-plus-dynamic representation in 11 of 12 matched GraphVAE comparisons. Semi-supervision helped consistently for the full-graph runs and the interface static runs, but not for the interface dynamic runs.

Within the full semi-supervised SD family, the reconstruction target also mattered. Averaged over latent dimensions 8, 16, and 32, the mean test RMSE was 2.4434 for `shared_static`, 2.4795 for `sd_static_plus_dynamic_all`, and 2.5150 for `sd_dynamic_all`. The corresponding mean test Pearson correlations were 0.4682, 0.4335, and 0.4221.

Model	Test RMSE	Test r	Test R^2	Test−Train RMSE
Full baseline, S	2.3379	0.5064	0.2146	0.8021
Full baseline, SD	2.4073	0.4802	0.1685	0.8414
Full GraphVAE, S, semi, $z = 32$	2.4292	0.4649	0.1537	0.7451
Full GraphVAE, SD, semi, shared static, $z = 8$	2.4364	0.4749	0.1478	1.1303
Interface baseline, S	2.5782	0.3451	0.0470	0.8090
Interface GraphVAE, S, semi, $z = 8$	2.6041	0.3402	0.0282	0.8018

Table 4.1: Best-performing models in the final matrix. GraphVAE metrics are those of the downstream ridge probe on latent means.

Comparison	Pairs	Mean RMSE gain for first option	Mean Pearson r gain for first option	Pair wins for first option
Full over interface	24	0.1005	0.0993	24 / 24
S over SD (shared static)	12	0.0254	0.0215	11 / 12
Semi over unsup, full S	3	0.1640	0.2497	3 / 3
Semi over unsup, full SD	9	0.1604	0.3339	9 / 9
Semi over unsup, interface S	3	0.0290	0.2436	3 / 3
Semi over unsup, interface SD	9	-0.0081	0.2766	3 / 9

Table 4.2: Matched GraphVAE comparisons. A positive RMSE gain means the first-named option has lower mean test RMSE.

4.3 Family-level train, validation, and test behaviour

Table 4.3 summarizes the main model families. The unsupervised VAE families had small train–test gaps but weak correlations on all splits. The semi-supervised families fit the training set more strongly. The strongest held-out result remained the full static supervised baseline.

4.4 Performance by interaction class

The matched 1437-complex analysis set contained 966 complexes in the broad *other* class, 366 antibody–antigen complexes, and 105 TCR-pMHC complexes. For subgroup analysis, the outer-test predictions were pooled over all 10 fits, so each complex contributes two held-out predictions, one from each repeat.

Table 4.4 shows the subgroup behaviour of the main headline models. The full static baseline remained the strongest of these models in all three interaction classes. The full static GraphVAE remained second, and the interface baseline remained weaker in every subgroup.

Family	Train r	Val r	Test r	g_r	Train RMSE	Test RMSE
Full baseline S	0.8225	0.5337	0.5064	0.3161	1.5358	2.3379
Full baseline SD	0.8077	0.5191	0.4802	0.3275	1.5659	2.4073
Full VAE unsup S	0.2486	0.1897	0.2055	0.0431	2.5734	2.6009
Full VAE unsup SD	0.1809	0.0967	0.1074	0.0736	2.6062	2.6398
Full VAE semi S	0.7494	0.4666	0.4551	0.2942	1.7385	2.4369
Full VAE semi SD	0.8218	0.4580	0.4413	0.3805	1.4744	2.4793
Interface VAE unsup S	0.1495	0.0794	0.0925	0.0570	2.6289	2.6402
Interface VAE unsup SD	0.1261	0.0382	0.0424	0.0838	2.6324	2.6538
Interface VAE semi S	0.7083	0.3356	0.3361	0.3722	1.8467	2.6112
Interface VAE semi SD	0.7685	0.3231	0.3189	0.4496	1.6479	2.6620

Table 4.3: Family-level mean train, validation, and test behaviour. Here $g_r = r_{\text{train}} - r_{\text{test}}$. Each row is averaged over all configurations in that family.

Subgroup	n	Full baseline RMSE	Full baseline r	Full GraphVAE RMSE	Full GraphVAE r	Interface baseline RMSE	Interface baseline r
Antibody–antigen	366	2.048	0.292	2.134	0.193	2.356	0.089
Other	966	2.501	0.430	2.584	0.394	2.675	0.312
TCR-pMHC	105	1.663	0.466	1.879	0.406	2.440	0.333

Table 4.4: Pooled outer-test subgroup metrics for the best overall model (`full_base_S`), the best GraphVAE (`full_S_semi_shared_static_z32`), and the best interface model (`iface_base_S`).

4.5 Summary

Four results dominate the final matrix. First, the full graph was better than the interface-restricted representation. Second, the dynamic summaries did not improve held-out performance over the static representation. Third, the strongest model was the direct supervised baseline on the full static graph. Fourth, the same model ordering remained in the antibody–antigen, TCR-pMHC, and other interaction classes.

Chapter 5

Discussion and Conclusions

On the 1437-complex evaluation set, the strongest model was the direct supervised baseline on the full static residue graph. The full view outperformed the interface-restricted view consistently, the molecular-dynamics-derived summaries did not improve held-out performance over the static feature set, and the direct supervised baseline outperformed the best GraphVAE configuration.

The graph-view comparison is unambiguous. The full graph won all 24 matched GraphVAE comparisons and both supervised baseline comparisons. In this implementation, the interface-restricted view removed useful context.

The dynamic comparison is specific to the implemented feature set: hydration summaries, Cα RMSF, torsion summaries, contact frequencies, distance summaries, and approximate inter-chain force proxies attached to a static residue graph. Under that encoding, the dynamic channels did not produce a held-out gain.

The GraphVAE learned usable latent representations, but the best predictive model was still the supervised baseline. On this dataset and with this encoder backbone, masked reconstruction and latent regularization did not yield the strongest held-out predictor.

The main limitations are the short trajectory length, the static graph topology, the use of time-aggregated dynamic features rather than an explicitly temporal model, and evaluation on one dataset family. A practical limitation is dataset scale: the processed broad table contained 2818 curated complexes, but the full MD and repeated-resampling study was run on a smaller compute-bounded analysis subset. PPB-Affinity reported better benchmark performance as the training fraction increased, so this restricted study size matters [3]. A second limitation is the split itself. The resampling was repeated, but it was not family-disjoint. Earlier protein–protein work explicitly removed homologous complexes to avoid optimistic evaluation, and protein–ligand studies have shown that structural overlap between training and test sets can inflate reported affinity-prediction performance [7, 18]. Natural extensions are therefore larger MD-backed study sets, longer simulations, models that operate directly on temporal graph sequences, and stricter split definitions.

Subtype heterogeneity also remained visible in the final sweep. On pooled outer-test predictions, the best overall model (`full_base_S`) gave Pearson $r = 0.292$ on antibody–antigen, $r = 0.430$ on the broad other class, and $r = 0.466$ on TCR-pMHC. The full static baseline remained the strongest configuration in all three interaction classes, and the interface-restricted models remained weaker in all three.

Within the implemented pipeline, short explicit-solvent molecular-dynamics summaries did not improve protein–protein binding affinity prediction over a static residue graph, and the strongest overall model was the direct supervised baseline on the full static graph.

Appendix A

Archived Approaches and Project Evolution

This appendix records the main branches that were explored seriously during the project but did not become part of the final reported pipeline. They are preserved under `ARCHIVE` in the repository and can also be browsed on GitHub at `https://github.com/RohanDevereux/Bridging/tree/main/ARCHIVE`.

A.1 Correction learning around fast affinity baselines

An early project idea was to predict a correction

$$\delta = \Delta G_{\text{exp}} - \Delta G_{\text{base}}$$

rather than predict ΔG directly. Two branches were explored in this form: a contact-based baseline using PRODIGY [13] and an endpoint free-energy baseline using MM/GBSA [12].

A.1.1 PRODIGY

The PRODIGY-related code is preserved under `ARCHIVE/PRODIGY`. This branch contains dataset parsing, chain-group normalization, estimate prefetching, and a runner that calls an external PRODIGY executable and parses the predicted ΔG [13].

On the preserved TCR subset runs, raw PRODIGY predictions were poor. For the common 13-complex test subset in the saved summaries, PRODIGY gave test RMSE 10.50 and test Pearson correlation -0.268. Some learned latent regressors reduced RMSE relative to PRODIGY on the overlapping test cases, but correction-to-PRODIGY was not competitive with direct latent regression.

The saved subset-specific baseline results also showed that the interaction classes behaved differently. On a TCR-only archived dataset, PRODIGY was essentially uncorre-

Archived subset	n	RMSE	MAE	Pearson r	Spearman ρ
TCR-pMHC	65	9.19	7.12	−0.051	−0.044
Antibody–antigen	70	2.33	1.80	0.243	0.236
Diverse100 mixed set	100	6.75	4.66	−0.175	−0.086

Table A.1: Saved subset-specific PRODIGY results from archived curated datasets. The TCR-pMHC subset was especially weak, while the small antibody–antigen subset was better but still modest.

Run	PRODIGY n	PRODIGY RMSE	PRODIGY r	Direct RMSE	Direct r	Co
`train80_test20_cpu`	13	10.50	−0.268	6.92	−0.018	
`setedge_e24`	13	10.50	−0.268	2.80	−0.500	

Table A.2: Representative archived PRODIGY-based regression runs on the TCR subset. Direct latent regression reduced RMSE relative to raw PRODIGY, but the correction-to-PRODIGY route was not competitive.

lated with experiment. On a small antibody–antigen archived dataset, the same baseline was better, though still modest. A mixed 100-complex subset was again poor.

A.1.2 MM/GBSA

The MM/GBSA branch is preserved under `ARCHIVE/MMGBSA`. It includes AmberTools-based input preparation, HPC launchers, prefetched estimate handling, and a runner that can execute MM/GBSA from a structure or trajectory [11, 12].

Across the shard files there were 163 rows with status `ok`. On the 163 rows that could be compared against the experimental PPB-Affinity target, the MM/GBSA estimates had Pearson correlation −0.127, Spearman correlation −0.096, RMSE 2458.69, and MAE 1398.31. The numerical scale was pathological: the median successful estimate was 466.38 kcal mol^{-1}, the 95th percentile was 5590.71 kcal mol^{-1}, and the maximum exceeded 13 000 kcal mol^{-1}, whereas the corresponding experimental free energies lay roughly between −21 and −4 kcal mol^{-1}.

These estimates were far outside the experimental scale and were not used further as a quantitative comparator.

A.2 The Cα contact-map branch

The oldest major modelling branch in the repository is the contact-map pipeline preserved under `ARCHIVE/CA_contact_map`. This branch converts selected interface residues into fixed two-dimensional tensors built from Cα–Cα contact and distance channels, then trains a convolutional variational autoencoder and downstream regression model on those tensors.

A contact map gives a fixed-size, image-like representation that is much easier to batch and process than a variable-size protein graph. That design choice has close relatives in

Quantity	Value
Successful rows	163
Comparable successful rows	163
Pearson correlation	-0.127
Spearman correlation	-0.096
RMSE	2458.69
MAE	1398.31
Median predicted MM/GBSA ΔG	466.38
95th percentile predicted MM/GBSA ΔG	5590.71
Maximum predicted MM/GBSA ΔG	13628.15

Table A.3: MM/GBSA shard results from the archived branch. The estimates are numerically far outside the scale of experimental protein–protein binding free energies, so the branch was not usable as a quantitative baseline.

Run	Test n	Direct-latent RMSE	Direct-latent Pearson r
`setedge_cpu`	8	3.03	-0.065
`setedge_e24`	12	2.80	-0.500
`setedge_e24_ols`	12	2.76	-0.527
`setedge_e24_seed1`	12	6.62	-0.124
`setedge_e24_seed4`	12	3.17	0.462
`setedge_semisup`	12	5.48	0.072
`allmd_setedge_e24`	12	4.23	0.087

Table A.4: Representative CA contact-map regression runs on the archived TCR subset. The branch could achieve lower RMSE than raw PRODIGY on some runs, but the results were unstable and often had weak or negative test correlation.

protein–ligand learning, where voxelized or tensorized structural inputs have been used successfully [16].

The saved summaries show lower RMSE than raw PRODIGY on some runs, but not a stable predictor. For example, the `setedge_e24` run gave test RMSE 2.80 against PRODIGY's 10.50 on the same saved subset, and the `setedge_e24_ols` variant gave test RMSE 2.76. The same runs often had negative test correlations, and a five-seed sweep of the `setedge_e24` regressor showed substantial instability: test RMSE ranged from 2.80 to 6.62 and test Pearson correlation ranged from -0.500 to 0.462. Semi-supervised training and correction-to-PRODIGY were not improvements in the saved results.

Once the graph pipeline existed, the contact-map representation looked too compressed: it discarded residue identity detail, side-chain context, and explicit graph structure, while still depending on a fixed interface selection and a fixed tensor ordering. The saved outputs also showed a branch that could sometimes reduce error but did not do so in a stable way. The residue-graph formulation retained more structural information and connected more naturally to the MD-derived features used in the final pipeline.

A.3 The PPB-inspired augmentation and spatiotemporal branch

The PPB-inspired deep-learning branch is preserved under `ARCHIVE/PPB_Affinity_augmentation`. It contains baseline, frame-level, and trajectory-level training code, together with an ensemble stage that combines those predictions, and a substantial vendored model stack under `ARCHIVE/PPB_Affinity_augmentation/src/BaselineModel`.

This path tried to use MD more directly by learning from frames and trajectories rather than reducing the trajectory to a small set of graph-level summaries. PPB-Affinity was introduced as a machine-learning resource for protein drug discovery [3], and recent protein–ligand work has shown that spatio-temporal learning on MD-derived data can be effective in some settings [17]. The motivation was to test whether that style of learning could transfer to protein–protein affinity prediction, where the interfaces are larger, the labelled benchmarks are less standardized, and the structural heterogeneity is broader [3–5].

This branch does not have a preserved final benchmark summary comparable with the later residue-graph pipeline. The intended training setup was computationally heavy, so the code was not run through to a comparable evaluation. It remains a plausible future direction.

A.4 Exploratory dataset curation utilities

The final archived workstream is `ARCHIVE/Data_curation`. These scripts cover earlier filtering decisions, partner-size prefetching, subset selection, diverse-sample construction, and older cleaning steps for intermediate datasets. They remain useful as provenance for the final dataset definition.

Appendix B

Complete Model Matrix

This appendix gives the complete 52-model evaluation matrix, sorted by mean test RMSE. The machine-readable version is available in `RESULTS/final_resample_matrix_summary.csv`. Here *Sup.* denotes supervision status and *Gap* denotes the mean test-minus-train RMSE gap.

Appendix B. Complete Model Matrix

Table B.1: Complete 52-model evaluation matrix, sorted by mean test RMSE. Metrics are means over 10 outer held-out fits.

View	Model	Mode	Sup.	Target	z	RMSE	r	R^2	Gap
Full	Baseline	S	sup	–	–	2.3379	0.5064	0.2146	0.8021
Full	Baseline	SD	sup	–	–	2.4073	0.4802	0.1685	0.8414
Full	GraphVAE	S	semi	shared static	32	2.4292	0.4649	0.1537	0.7451
Full	GraphVAE	SD	semi	shared static	8	2.4364	0.4749	0.1478	1.1303
Full	GraphVAE	S	semi	shared static	8	2.4407	0.4507	0.1451	0.6921
Full	GraphVAE	S	semi	shared static	16	2.4407	0.4499	0.1438	0.6580
Full	GraphVAE	SD	semi	static+dynamic	16	2.4419	0.4503	0.1437	0.8952
Full	GraphVAE	SD	semi	shared static	32	2.4423	0.4669	0.1443	1.2169
Full	GraphVAE	SD	semi	shared static	16	2.4516	0.4629	0.1354	1.1312
Full	GraphVAE	SD	semi	static+dynamic	32	2.4687	0.4411	0.1233	0.8527
Full	GraphVAE	SD	semi	dynamic only	32	2.4740	0.4508	0.1231	1.0153
Full	GraphVAE	SD	semi	dynamic only	8	2.5168	0.4202	0.0920	0.9580
Full	GraphVAE	SD	semi	static+dynamic	8	2.5277	0.4092	0.0818	0.7819
Full	GraphVAE	SD	semi	dynamic only	16	2.5543	0.3954	0.0629	1.0628
Interface	Baseline	S	sup	–	–	2.5782	0.3451	0.0470	0.8090
Interface	Baseline	SD	sup	–	–	2.5829	0.3347	0.0463	0.6954
Full	GraphVAE	S	unsup	shared static	32	2.5906	0.2271	0.0401	0.0617
Interface	GraphVAE	S	semi	shared static	8	2.6041	0.3402	0.0282	0.8018
Full	GraphVAE	S	unsup	shared static	16	2.6052	0.1910	0.0303	0.0194
Full	GraphVAE	S	unsup	shared static	8	2.6069	0.1984	0.0286	0.0016
Interface	GraphVAE	S	semi	shared static	32	2.6113	0.3364	0.0207	0.7354
Interface	GraphVAE	S	semi	shared static	16	2.6180	0.3316	0.0166	0.7561
Interface	GraphVAE	SD	semi	static+dynamic	16	2.6238	0.3252	0.0146	0.7597
Full	GraphVAE	SD	unsup	shared static	16	2.6313	0.1259	0.0102	0.0265
Full	GraphVAE	SD	unsup	shared static	8	2.6322	0.1249	0.0103	0.0124
Interface	GraphVAE	S	unsup	shared static	32	2.6324	0.1211	0.0102	0.0118
Full	GraphVAE	SD	unsup	static+dynamic	32	2.6328	0.1255	0.0093	0.0610
Full	GraphVAE	SD	unsup	dynamic only	32	2.6353	0.1100	0.0074	0.0364
Full	GraphVAE	SD	unsup	shared static	32	2.6365	0.1813	0.0039	0.1021
Interface	GraphVAE	SD	semi	dynamic only	16	2.6366	0.3298	0.0032	0.9460
Interface	GraphVAE	S	unsup	shared static	8	2.6427	0.0790	0.0025	0.0062
Full	GraphVAE	SD	unsup	dynamic only	16	2.6441	0.0981	0.0013	0.0292
Interface	GraphVAE	S	unsup	shared static	16	2.6454	0.0774	0.0003	0.0157
Full	GraphVAE	SD	unsup	static+dynamic	8	2.6454	0.0719	0.0006	0.0048
Interface	GraphVAE	SD	semi	shared static	16	2.6455	0.3362	-0.0015	1.1836

View	Model	Mode	Sup.	Target	z	RMSE	r	R^2	Gap
Full	GraphVAE	SD	unsup	dynamic only	8	2.6461	0.0706	0.0000	0.0013
Interface	GraphVAE	SD	unsup	shared static	8	2.6473	0.0618	-0.0010	0.0129
Interface	GraphVAE	SD	unsup	static+dynamic	8	2.6481	0.0552	-0.0016	0.0056
Interface	GraphVAE	SD	unsup	shared static	32	2.6530	0.0896	-0.0051	0.0451
Interface	GraphVAE	SD	unsup	static+dynamic	32	2.6534	0.0268	-0.0056	0.0222
Full	GraphVAE	SD	unsup	static+dynamic	16	2.6542	0.0582	-0.0060	0.0286
Interface	GraphVAE	SD	unsup	dynamic only	8	2.6552	0.0293	-0.0071	0.0097
Interface	GraphVAE	SD	unsup	dynamic only	16	2.6554	0.0235	-0.0070	0.0151
Interface	GraphVAE	SD	semi	dynamic only	8	2.6557	0.3279	-0.0116	1.0207
Interface	GraphVAE	SD	unsup	static+dynamic	16	2.6557	0.0209	-0.0073	0.0127
Interface	GraphVAE	SD	unsup	shared static	16	2.6575	0.0395	-0.0088	0.0353
Interface	GraphVAE	SD	unsup	dynamic only	32	2.6589	0.0347	-0.0099	0.0347
Interface	GraphVAE	SD	semi	shared static	8	2.6684	0.3270	-0.0217	1.1249
Interface	GraphVAE	SD	semi	shared static	32	2.6705	0.3185	-0.0230	1.2466
Interface	GraphVAE	SD	semi	static+dynamic	8	2.6743	0.3039	-0.0243	0.8622
Interface	GraphVAE	SD	semi	dynamic only	32	2.6855	0.3076	-0.0322	1.0470
Interface	GraphVAE	SD	semi	static+dynamic	32	2.6974	0.2941	-0.0411	0.9357

Appendix C

Repository Layout

The source code for this thesis is hosted at `https://github.com/RohanDevereux/Bridging`. The active molecular-dynamics preparation and simulation code is under `src/bridging/MD`. The residue-graph preparation code is under `src/bridging/graphvae/prep`. The GraphVAE, supervised baseline, and latent-ridge evaluation code are under `src/bridging/graphvae/ml`. The command-line runners used for the final evaluation are under `src/bridging/graphvae/runners`. Shared affinity-conversion and dataset utilities are under `src/bridging/utils`.

The curated datasets used in the main pipeline are under `src/bridging/processedData`. The compact summary of the final 52-configuration matrix is `RESULTS/final_resample_matrix_summary.csv`. The saved subgroup analysis derived from the raw outer-fold prediction files is under `RESULTS/final_resample_subgroups`. The script that rebuilds those subgroup summaries from extracted raw predictions is `src/bridging/graphvae/tools/export_final_resample_subgroups.py`. Archived branches discussed in Appendix A are under `ARCHIVE`.

Bibliography

[1] Ashley E. Modell, Sarah L. Blosser, and Paramjit S. Arora. Systematic targeting of protein-protein interactions. *Trends in Pharmacological Sciences*, 37(8):702–713, 2016. doi: 10.1016/j.tips.2016.05.008.

[2] Zhongliang Guo and Rui Yamaguchi. Machine learning methods for protein-protein binding affinity prediction in protein design. *Frontiers in Bioinformatics*, 2:1065703, 2022. doi: 10.3389/fbinf.2022.1065703.

[3] Huaqing Liu, Peiyi Chen, Xiaochen Zhai, Ku-Geng Huo, Shuxian Zhou, Lanqing Han, and Guoxin Fan. Ppb-affinity: Protein-protein binding affinity dataset for ai-based protein drug discovery. *Scientific Data*, 11:1316, 2024. doi: 10.1038/s41597-024-03997-4.

[4] Renxiao Wang, Xiaolu Fang, Yipin Lu, and Shaomeng Wang. The PDBbind database: Collection of binding affinities for protein–ligand complexes with known three-dimensional structures. *Journal of Medicinal Chemistry*, 47(12):2977–2980, 2004. doi: 10.1021/jm030580l.

[5] Panagiotis L. Kastritis, Iain H. Moal, Howook Hwang, Zhiping Weng, Paul A. Bates, Alexandre M. J. J. Bonvin, and Joël Janin. A structure-based benchmark for protein-protein binding affinity. *Protein Science*, 20(3):482–491, 2011. doi: 10.1002/pro.580.

[6] Till Siebenmorgen and Martin Zacharias. Computational prediction of protein–protein binding affinities. *Wiley Interdisciplinary Reviews: Computational Molecular Science*, 10(3):e1448, 2020. doi: 10.1002/wcms.1448.

[7] Iain H. Moal, Rudi Agius, and Paul A. Bates. Protein-protein binding affinity prediction on a diverse set of structures. *Bioinformatics*, 27(21):3002–3009, 2011. doi: 10.1093/bioinformatics/btr513.

[8] Yong Xiao Yang, Pan Wang, and Bao Ting Zhu. Binding affinity prediction for antibody-protein antigen complexes: A machine learning analysis based on interface and surface areas. *Journal of Molecular Graphics and Modelling*, 118:108364, 2023. doi: 10.1016/j.jmgm.2022.108364.

[9] Glenn M. Torrie and John P. Valleau. Nonphysical sampling distributions in Monte Carlo free-energy estimation: Umbrella sampling. *Journal of Computational Physics*, 23(2):187–199, 1977. doi: 10.1016/0021-9991(77)90121-8.

[10] Zoe Cournia, Bryce Allen, and Woody Sherman. Relative binding free energy calculations in drug discovery: Recent advances and practical considerations. *Journal of Chemical Information and Modeling*, 57(12):2911–2937, 2017. doi: 10.1021/acs.jcim.7b00564.

[11] Iva Massova and Peter A. Kollman. Combined molecular mechanical and continuum solvent approach (MM-PBSA/GBSA) to predict ligand binding. *Perspectives in Drug Discovery and Design*, 18:113–135, 2000.

[12] Samuel Genheden and Ulf Ryde. The MM/PBSA and MM/GBSA methods to estimate ligand-binding affinities. *Expert Opinion on Drug Discovery*, 10(5):449–461, 2015. doi: 10.1517/17460441.2015.1032936.

[13] Anna Vangone and Alexandre M. J. J. Bonvin. Contacts-based prediction of binding affinity in protein-protein complexes. *eLife*, 4:e07454, 2015. doi: 10.7554/eLife.07454.

[14] Sandra Romero-Molina, Yasser B. Ruiz-Blanco, Joel Mieres-Perez, Mirja Harms, Jan Münch, Michael Ehrmann, and Elsa Sanchez-Garcia. Ppi-affinity: A web tool for the prediction and optimization of protein-peptide and protein-protein binding affinity. *Journal of Proteome Research*, 21(8):1829–1841, 2022. doi: 10.1021/acs.jproteome.2c00020.

[15] Jun Xie, Youli Zhang, Ziyang Wang, Xiaocheng Jin, Xiaoli Lu, Shengxiang Ge, and Xiaoping Min. Ppi-graphomer: Enhanced protein-protein affinity prediction using pretrained and graph transformer models. *BMC Bioinformatics*, 26:116, 2025. doi: 10.1186/s12859-025-06123-2.

[16] Marta M. Stepniewska-Dziubinska, Piotr Zielenkiewicz, and Pawel Siedlecki. Development and evaluation of a deep learning model for protein–ligand binding affinity prediction. *Bioinformatics*, 34(21):3666–3674, 2018. doi: 10.1093/bioinformatics/bty374.

[17] Pierre-Yves Libouban, Camille Parisel, Maxime Song, Samia Aci-Sèche, Jose C. Gómez-Tamayo, Gary Tresadern, and Pascal Bonnet. Spatio-temporal learning from molecular dynamics simulations for protein–ligand binding affinity prediction. *Bioinformatics*, 41(8):btaf429, 2025. doi: 10.1093/bioinformatics/btaf429.

[18] David Graber, Peter Stockinger, Fabian Meyer, Siddhartha Mishra, Claus Horn, and Rebecca Buller. Resolving data bias improves generalization in binding affin-

ity prediction. *Nature Machine Intelligence*, 7(10):1713–1725, 2025. doi: 10.1038/s42256-025-01124-5.

[19] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. arXiv preprint arXiv:1312.6114, 2014. URL `https://arxiv.org/abs/1312.6114`.

[20] Thomas N. Kipf and Max Welling. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308*, 2016. URL `https://arxiv.org/abs/1611.07308`.

[21] Hao Tian, Xi Jiang, Francesco Trozzi, Sian Xiao, Eric C. Larson, and Peng Tao. Explore protein conformational space with variational autoencoder. *Frontiers in Molecular Biosciences*, 8:781635, 2021. doi: 10.3389/fmolb.2021.781635.

[22] OpenMM Developers. Pdbfixer. `https://github.com/openmm/pdbfixer`, 2026. Accessed 2026-03-17.

[23] Peter Eastman, Raimondas Galvelis, Raúl P. Peláez, Charlline R. A. Abreu, Stephen E. Farr, Emilio Gallicchio, et al. Openmm 8: Molecular dynamics simulation with machine learning potentials. *The Journal of Physical Chemistry B*, 128 (1), 2024. doi: 10.1021/acs.jpcb.3c06662.

[24] James A. Maier, Carmenza Martinez, Koushik Kasavajhala, Lauren Wickstrom, Karl E. Hauser, and Carlos Simmerling. ff14sb: Improving the accuracy of protein side chain and backbone parameters from ff99sb. *Journal of Chemical Theory and Computation*, 11(8):3696–3713, 2015. doi: 10.1021/acs.jctc.5b00255.

[25] Lee-Ping Wang, Todd J. Martinez, and Vijay S. Pande. Building force fields: An automatic, systematic, and reproducible approach. *The Journal of Physical Chemistry Letters*, 5(11):1885–1891, 2014. doi: 10.1021/jz500737m.

[26] Giulia Crocioni, Dani L. Bodor, Coos Baakman, Farzaneh M. Parizi, Daniel-T. Rademaker, Gayatri Ramakrishnan, Sven A. van der Burg, Dario F. Marzella, João M. C. Teixeira, and Li C. Xue. Deeprank2: Mining 3d protein structures with geometric deep learning. *Journal of Open Source Software*, 9(94):5983, 2024. doi: 10.21105/joss.05983.

[27] Robert T. McGibbon, Kyle A. Beauchamp, Matthew P. Harrigan, Christoph Klein, Jason M. Swails, Carlos X. Hernández, Christian R. Schwantes, Lee-Ping Wang, Thomas J. Lane, and Vijay S. Pande. Mdtraj: A modern open library for the analysis of molecular dynamics trajectories. *Biophysical Journal*, 109(8):1528–1532, 2015. doi: 10.1016/j.bpj.2015.08.015.

[28] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with pytorch geometric. *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. URL `https://arxiv.org/abs/1903.02428`.

[29] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=ryGs6iA5Km`.

[30] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. Strategies for pre-training graph neural networks. *International Conference on Learning Representations*, 2020. URL `https://openreview.net/forum?id=HJlWWJSFDH`.

[31] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL `https://jmlr.org/papers/v12/pedregosa11a.html`.